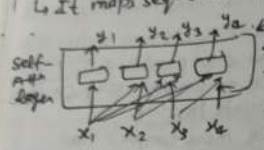
Transformer (~~Use it all~~ ~~Feat~~)

↳ It maps sequences of I/P vectors to seq. of O/P vectors of same length.

(Causal or masked) self-Attention model.



← Processing happens in $\|^{el}$.

→ Comparison ⇒ $\text{score}(x_i, x_j) = x_i \cdot x_j$. $\forall j \leq i$

→ $\alpha_{ij} = \text{softmax}(\text{score}(x_i, x_j))$

→ $y_i = \sum_{j \leq i} \alpha_{ij} x_j$.

⇒ $\text{Self-Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$.

Source: LLM University by Cohera

(*) What are Word & Sentence Embeddings?

⇒ Sentence & word embeddings are bread & butter of language models.

The main task of NLP is to understand human language. However, there is a big disconnection there. Humans speak in words & sentences, but computers only understand & process numbers. How can we turn words & sentences into numbers in a coherent way? An assignment of words to numbers is called a word embedding. We can think of a word embedding as an assignment of scores to the words, with some nice properties.

(*) What is a Word Embedding?

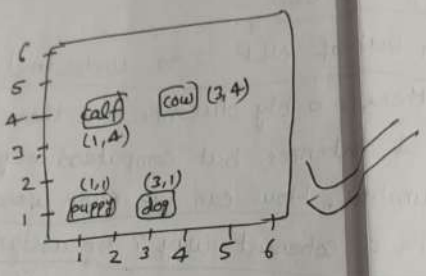
↳ Words that are similar should correspond to points that are close by (or equivalently, to scores that are similar).

↳ Words that are different should correspond to points that are far away (or equivalently, to scores that are significantly different).

(*) Word Embeddings capture features of the word.

↳ Word embeddings not only capture word similarity but they also capture other properties of the language.

In language, words can be combined to get more complicated concepts. In mathematics, numbers can be added or subtracted to get other numbers. Could we build a word embedding that captures relations b/w words, as relations b/w numbers?



↳ here, puppy, dog, calf & cow makes a rectangle.

puppy : dog :: calf : cow

↳ The path from the word puppy to the word dog is same as the path from the calf to the word cow.

↳ The analogy "A dog is to a cow like a puppy is to calf" is also captured in this rectangle.

→ The main property of word embeddings that is in effect here is that the two axes (vertical & horizontal) represent different things. If you look carefully, moving towards the right turns the puppy into a dog & the calf into a cow, which is an increase in age. Likewise, moving upwards turns a puppy into a calf and a dog into a cow, which is an increase in the size of the animals.

It seem
the words
age & size
is locating
the vert
imagine
somewhere
was a
go some

↳ A good
only age
words. Sin
then a ge
coordinates
embedding
with each
coordinat
the vecto
the neu
Some of
properti
Some ma
Some other
human

It seems that this embedding is understanding that the words in it have 2 main properties or features: age & size. Furthermore, it seems that the embedding is locating age in the horizontal axis & size in the vertical axis. In that case, where would you imagine that the word "whale" goes? Probably, somewhere above the word "cow". And if there was a word for "really old dog"? That word would go somewhere to the right of the word "dog".

↳ A good word embedding would be able to capture not only age & size, but also many more features of the words. Since each feature is one new axis, or coordinate, then a good embedding must have many more than two coordinates assigned to every word. One of the Cohere embeddings, for example, has 1024 coordinates associated with each word. These rows of 1024 (or however many) coordinates are called vectors, so we often talk about the vector corresponding to a word, and to each of the numbers inside a vector as a coordinate. Some of these coordinates may represent important properties of the word, such as age, gender, size. Some may represent combinations of properties. But Some others may represent obscure properties that a human may not ~~at~~ be able to understand.

But all in all, a word embedding can be seen as a good way to translate human language (words) into computer language (numbers), so that we can start training ML models with these numbers.

Sentence Embeddings

So, word embeddings seem to be pretty useful, but in reality, human language is much more complicated than simply a bunch of words put together. How would one be able to represent a sentence? How about the sums of scores of all the words?

For ex., say we have a word embedding that assigns the following scores to these words:

No : [1, 0, 0, 0]
I : [0, 2, 0, 0]
Am : [-1, 0, 1, 0]
Good : [0, 0, 1, 3]

Then, the sentence "No, I am good!" corresponds to the vector [0, 2, 2, 3]. However, the sentence "I am no good" will also correspond to the vector [0, 2, 2, 3]. This is not a good thing, since the computer understands these 2 sentences in the exact same way, yet the

they are quite
Therefore, we ne
account the c
language and

A sentenc
except it asso
of numbers
that it sati
For instan
similar vec
different vec
coordinates o
the sentenc

Sentenc

they are quite different, almost opposite sentences. Therefore, we need better embeddings that take into account the order of words, the semantics of the language and the actual meaning of the sentence.

A sentence embedding is just like a word embedding except it associates every sentence with a vector full of numbers, in a coherent way. By coherent, I mean that it satisfies similar properties as a word embedding. For instance, similar sentences are assigned to similar vectors, different sentences are assigned to different vectors & most importantly, each of the coordinates of the vector identifies some properties of the sentence.

Sentence Embeddings

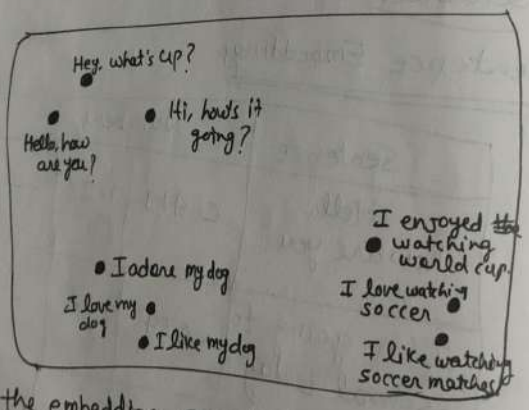
Sentence	Numbers			
Hello, how are you?	0.419	1.28	---	-0.06
I'm going to school today	-0.74	1.02	---	1.35
---	---	---	---	---
Once upon a time	-0.82	-1.32	---	0.32

The cohere embedding does just this. Using transformers, attention mechanisms & other cutting edge algo^ms, this embedding sends every sentence to a vector formed by 4096 numbers; & this embedding works really well.

⇒ How to use these Embeddings?

use Cohere dashboard. → Embed tab. & type sentences

↳ The results come out as vectors with 4096 entries for each sentence. These are obviously hard to visualize but there is a way to bring them down to 2 entries per sentence in order to be easily visualized.



↳ Notice that the embedding seemed to capture the essence of the sentences & there are 3 clear clusters of sentences.

(*) What is

For LLMs, it
sentences are
prob^m but luc
very helpful

↳ In a nutshel
a list of n
that semant
mathematical
by this? For
vectors. & 2
But most im
corresponding
property o
can be un
size, gend
only be

Sent
as they ar
sentence,
carry im
One of th
of length
each

(*) What is similarity b/w sentences.

For LLMs, it is crucial to know when 2 words or 2 sentences are similar or different. This can be a hard prob^{m} but luckily, word & sentence embeddings are very helpful for this task.

↳ In a nutshell, a word embedding is an assignment of a list of numbers (vector) to every word, in a way that semantic properties of the word translate into mathematical properties of numbers. What do we mean by this? For ex., 2 similar words will have similar vectors. & 2 diff. words will have diff vectors. But most importantly, each entry in the vector corresponding to a word keeps track of some property of the word. Some of these properties can be understandable to humans such as age, size, gender etc, but some other could potentially only be understood by the computer.

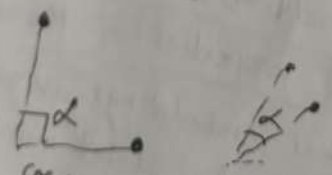
Sentence embeddings are even more powerful, as they assign a vector of numbers to each sentence, in a way that these numbers also carry important properties of the sentence. One of the Cohere embeddings assigns a vector of length 4096 (a list of 4096 numbers) to each sentence.

↳ There are 2 types of similarities: dot product similarity & cosine similarity. Both are very similar & very useful to determine if 2 words (or sentences) are similar.

↳ In dot product, we multiply vectors elementwise & then add. In Cosine Similarity, we see the angles b/w 2 sentences.

↳ Euclidean distance is a good measure for similarity [very far movies are less similar]

& it does the opposite.. Now, in order to tweak this metric, let's look at the angle b/w the rays from the origin & each sentence. Notice that this angle is small if the points are close to each other, & large if the points are far away from each other. Now we need the help of another function, the cosine. The cosine of angles close to zero is close to 1. & as the angle grows, the cosine decreases.. This is exactly what we need. Therefore, we define the cosine distance as the cosine of the angle formed by the 2 rays going from the origin, to the 2 sentences.



$\cos\alpha = 0$
Similarity = 0

$\cos\alpha = 0.98$
Similarity = 0.98.

⇒ Real-Life

import n
import se
import o
from skl
import
co = co

Consider
texts =

↳ One would
similarity scor
have a very
two.

↳ To get the
the Embed
parameters
- texts : o
- model : u

⇒ input_type

→ Real-Life Example

```
import numpy as np
import seaborn as sns
import altair as alt
from sklearn.metrics.pairwise import cosine_similarity
import cohere
co = cohere.Client("COHERE_API_KEY") # Your Cohere API key.

# Consider the following 3 sentences.
texts = ["I like to be in my house",
         "I enjoy staying home",
         "the isotope 238u decays to 206pb"]
```

↳ One would expect the first 2 sentences to have a high similarity score when compared to each other. & 3rd one to have a very low similarity score when compared to the other two.

↳ To get the corresponding sentence embeddings, we call the Embed endpoint with co.embed(). we supply 3 parameters:

- texts : our list of sentences
- model : we use embed-english-v3.0, Cohere's latest English-only embeddings model to generate embeddings
- input_type : we use search_document to indicate that we intend to use embeddings for search use

↳ response = co.embed(
texts = texts,
model = 'embed-english-v3.0',
input_type = 'search_document'
)

↳ The embeddings are stored in the embeddings value of the response. After getting the embeddings, we separate them by sentence & print the values.

⇒ embeddings = response.embeddings

[sentence 1, sentence 2, sentence 3] = embeddings

print("Embedding for sentence 1", np.array(sentence1))
print(" " " " 2", " " (" 2))
" " " " " 3", " " (" 3))

↳ Note that the embeddings are vectors (lists) of 1024 numbers, so they are truncated here.

⇒ Dot Product Similarity.

print("Similarity b/w sentence 1 & 2:", np.dot(sentence1, sentence2))
" " " " 1 & 3: " " " " 3))
" " " " 2 & 3, " " " 2 3))

⇒ Cosine similarity.

print("Cosine similarity b/w sentence 1 & 2:", cosine_similarity([sentence1], [sentence 2])[0][0])

& same for others.

⇒ Dot Produc
The reason fo
(each vector
is not norm
would gi

(*) What

A huge re
can be used
encountered.
sentence in
to use. T
word
that have
assigns a ve
the same
What if yo
Here is w

Att
apart wh
(which the
word e